

Proyecto Cursada

Alumnos: Cid Nicolás - Bertotto Tomás

6A)

A

B

CRs

CR_A
factorGlobal = 10

main

RA main
ref args
input1
input2
objetoC
resultado1

RA constructor A
PR
ED
valorBase

RA obtenerEstado() en A
PR
ED
this

RA calcular(int limite) en A
PR
ED
this
limite
resultado
i

RA constructor B
PR
ED
valorBase
multiplicador

RA procesarSimple(int x) en B
PR
ED
this
x
base

RA calcularConFor(int repeticiones) en B
PR
ED
this
repeticiones
total
j

C

RA constructor C
PR
ED
valorBase
multiplicadorB

RA obtenerEstado() en C
PR
ED
this

RA procesarComplejo(int x, int y) en C
PR
ED
this
x
y

INSTs

VTs

oa1 : class A
ref a VT_A
int valorBase

VT_A
ref a calcular(int limite) [en A]
ref a obtenerEstado() [en A]
ref a VT_Object

ob1 : class B
ref a VT_B
int valorBase
int multiplicador

VT_B
ref a calcular(int limite) [en A]
ref a obtenerEstado() [en A]
ref a procesarSimple(int x) [en B]
ref a calcularConFor(int repeticiones) [en B]
ref a VT_A

oc1 : class C
ref a VT_C
int valorBase
B objetoB

VT_C
ref a calcular(int limite) [en A]
ref a obtenerEstado() [en C]
ref a procesarComplejo(int x, int y) [en C]
ref a VT_A

6B) Realizamos el mecanismo iterativo vía el uso de dos Jump: uno condicional (JumpT) y otro incondicional (Jump). El primero de éstos evalúa la condición del “while” acorde al resultado: o bien entra a ejecutar las instrucciones correspondientes del cuerpo del while (i.e. el PC se actualiza solo de forma secuencial) o bien actualiza el PC forzándolo a salir a la primera instrucción fuera del while. En el caso del Jump incondicional se utiliza dentro de las instrucciones del while para comenzar una nueva iteración (i.e. vuelve al JumpT inicial).

Respecto a la implementación del “For” notamos que son **similares**, ambas poseen un jump condicional que evalúa si el PC continúa secuencialmente o sale de la estructura de control, y uno incondicional para volver al comienzo de la misma. Observamos que dicha similitud se corresponde a cómo las estructuras de control “while” y “for” funcionan en Java. Éstas últimas **dependen de una condición explícita** (a diferencia de, por ejemplo, un “for” en Pascal) y difieren, a nivel del lenguaje Java, en que la variable de control **se actualiza sola** en el *for*, mientras que en el caso del *while* debe agregarse una instrucción específica que lo haga. No obstante, la traducción a simplON resultante requiere una instrucción que incremente la variable de control en ambos casos. **Denotamos** con esto último la capacidad de abstracción del lenguaje en donde la traducción a simplON es similar **pero** a nivel del lenguaje dichas estructuras no sólo difieren sintácticamente sino que “ahorran” agregar una instrucción por parte del programador como lo es incrementar la variable de control.

Aclaración: las condiciones usando JumpT **se encuentran en su mayoría negadas** y por lo tanto “espejadas” respecto al código fuente en java.

6C) Consideramos que para realizar una traducción del mecanismo de selección múltiple **depende de cómo el lenguaje defina semánticamente a ésta estructura de control**. Identificamos dos escenarios posibles: selección múltiple **mutuamente excluyente** y **no mutuamente excluyente**. En el **primer caso** la traducción quedaría formada por varios jump condicionales que evalúen cada caso posible y para cada uno de ellos: el **código correspondiente** y un **jump incondicional** que salga del código de ésta estructura de control. Para el otro caso, sería ligeramente distinto en dónde para cada cada jump condicional cada uno tenga: el **código asociado** y un **jump incondicional** que **o bien** fuerce el PC al código asociado de la siguiente condición **o** un salto afuera de ésta estructura de control si el lenguaje admite el uso de “*break*”.

6D) La traducción de expresiones booleanas van a diferir en cómo se evalúa la expresión siendo con o sin cortocircuito. Para el caso de que la expresión se evalúe **con** cortocircuito se traduce en: una instrucción individual para cada condición (JumpT) y su actualización del PC. Para el restante caso solamente alcanza con una sola instrucción. Véase en *procesarComplejo*:

Ejemplo CON cortocircuito

Expresión: $x > 0 \ \&\& \ y < 100$

Traducción:

```
JumpT finCondicionalProcComC, !(D[Actual + 3] > 0) %sale si se cumple la negacion
JumpT finCondicionalProcComC, !(D[Actual + 4] < 100) %sale si se cumple la negacion
    %Codigo THEN (si no salta antes entra secuencialmente)
    %...

finCondicionalProcComC
```

Ejemplo ficticio SIN cortocircuito

Expresión: $x > 0 \ \& \ y < 100$

Traducción:

```
JumpT finCondicionalProcComC, !(D[actual+3] > 0 & D[actual+4] < 100) % sale/va al ELSE si se
cumple la negación
    %Codigo THEN(si no salta antes entra secuencialmente)
```

6E)

Llamada: this.<getObj1()>.<getObj2()>.<metodoObj2()>

CONVENCIÓN de NOTACIÓN “this” es de TIPO: “A”

Si tenemos en cuenta este tipo de llamadas entendemos que this (A) está en relación de **dependencia** con la clase del objeto 1 y este último con el objeto 2.

Entonces para poder realizar llamadas encadenadas de este tipo, consideramos la siguiente secuencia de pasos vistos desde la unidad llamadora:

1. Reservar en la memoria de datos un lugar para lo que retornen las unidades encadenadas (i.e. se reutilizará dicha locación)
2. Crear RA de GetObj1 (PR, ED, **This(A)**)
3. Jump GetObj1
4. (unidad **llamada** destruye el RA de GetObj1 y vuelve el control a la unidad **llamadora**)
5. **Reutiliza** en la memoria de datos el lugar utilizado previamente para lo que retornó *getObj1* para lo que retorne *getObj2*
6. Crear RA de GetObj2 (PR, ED, This(**TipoObj1**))
7. Jump GetObj2
8. (unidad **llamada** destruye el RA de GetObj2 y vuelve el control a la unidad **llamadora**)
9. (opcional) **Reutiliza** en la memoria de datos el lugar utilizado previamente para lo que retornó *getObj2* para lo que retorne *metodoObj2*
10. Crear RA de metodoObj2 (PR, ED, This(**TipoObj2**))
11. Jump metodoObj2
12. (unidad **llamada** destruye el RA de metodoObj2 y vuelve el control a la unidad **llamadora**)